



Local Language Models for Spoken Rover Task Planning with Clarification Dialogue

by

U. V. S. Rosant

*Research assignment presented in partial fulfilment of the
requirements for the degree of Bachelor of Engineering
(Electrical and Electronic) in the Faculty of Engineering at
Stellenbosch University*

Supervisor: Prof. TODO Surname

2026



Someone.

ACKNOWLEDGEMENTS

* Supervisor X.

* Funding Y.

“ *A very long quote*
from someone very important. ”

– Mr M

DECLARATION

By submitting this research assignment electronically, I declare that the entirety of the work contained therein is my own, original work, that I am the sole author thereof (save to the extent explicitly otherwise stated), that reproduction and publication thereof by Stellenbosch University will not infringe any third party rights and that I have not previously in its entirety or in part submitted it for obtaining any qualification.

Date: November 2026
.....

Copyright © 2026 Stellenbosch University
All rights reserved.

ABSTRACT

UITTREKSEL

TABLE OF CONTENTS

Abstract	iii
Uittreksel	iv
Table of contents	v
List of figures	vi
List of tables	vi
Nomenclature	viii
Acronyms and abbreviations	ix
1. Introduction	1
1.1. Background	1
1.2. Problem statement	2
1.3. Research questions	3
1.4. Objectives	3
1.5. Scope and limitations	3
1.6. Report outline	4
2. Literature Study	5
2.1. Language models as robot task planners	5
2.2. Grounding plans in a bounded action set	7
2.3. Clarification and ambiguity	9
2.4. Locally hosted models	11
2.5. Compositional depth and hallucination	12
2.6. Speech recognition for local languages	13
2.7. Research gap	14
3. System Design	15
3.1. User and functional requirements	15
3.2. System architecture	15

3.2.1. Off-board inference and the thin-client split	15
3.2.2. Two-rate control: waypoint intents versus the reactive layer	15
3.3. The capability vocabulary	15
3.4. Clarification loop design	15
3.5. Navigation as instrumentation	15
3.6. Summary of design decisions	15
4. Implementation	16
4.1. Speech-to-text subsystem	16
4.2. Planner	16
4.3. Vision-language verification	16
4.4. Clarification manager	16
4.5. Rover client and reactive layer	16
4.6. Communication protocol and dashboard	16
4.7. Logging and measurement instrumentation	16
5. Experimental Evaluation	17
5.1. Experimental methodology	17
5.1.1. Command corpus design	17
5.1.2. Evaluation metrics	17
5.2. Plan correctness under vague commands	17
5.3. Executability on the physical chassis	17
5.4. Compositional depth and hallucination onset	17
5.5. Reference comparison against cloud models	17
5.6. Discussion	17
6. Conclusion	18
6.1. Answers to the research questions	18
6.2. Contributions	18
6.3. Limitations	18
6.4. Recommendations for future work	18
References	19
A. Appendix title goes here	21

LIST OF FIGURES

2.1.	Process overview of RAP generation before rover departures	7
2.2.	SayCan combining language-model relevance scores with value-function afford- ance scores to select the next skill, reproduced from Ahn et al. [6].	8
2.3.	The KnowNo pipeline: an LLM planner proposes next steps with confidence scores, conformal prediction narrows these to a calibrated prediction set, and the rover asks for help whenever that set contains more than one option. Reproduced from Ren et al. [2].	10
2.4.	The introspective planning pipeline: a knowledge base of explained past examples is built offline, and at deployment the most similar example is retrieved to inform the LLM’s confidence for each candidate step. Reproduced from Liang et al. [3].	12

LIST OF TABLES

NOMENCLATURE

VARIABLES AND FUNCTIONS

CONSTANTS

c A constant.

FUNCTIONS

f A function.

VARIABLES

x A variable.

ACRONYMS AND ABBREVIATIONS

API

CPU

FPS

HRI Human-Robot Interaction

IMU

LLM Large Language Model

SLAM

STT Speech-to-Text

VLM Vision-Language Model

DEFINITIONS

S

STUDENT is an entity needing a thesis to transcend the state of being a student.

INTRODUCTION

1.1 BACKGROUND

Having a robot that takes spoken instructions and removing the two things that make most mobile robots hard to operate day to day: a screen to navigate, and an operator trained on its command set. For a small indoor rover this matters even more, since the tasks it can reasonably be asked to do (check whether a door is open, look at what’s on a shelf, go and see who’s in the next room) are exactly the kind of thing a person would rather say out loud than type into an app.

Until recently, building this meant hand-coding a fixed vocabulary of recognized phrases, each mapped to a specific behavior. Large language models change that. Given a natural-language command and a description of what the robot can do, an LLM can decompose an instruction into a sequence of primitive actions without the vocabulary being enumerated in advance. Hori et al. demonstrate this for long-horizon tasks specifically, using a dialogue-based process where the model asks questions to fill in what a one-shot instruction leaves out before committing to a plan [1].

Two problems follow directly from handing planning to a language model instead of a fixed rule set. First, spoken commands are often genuinely ambiguous. “Look at the plant” is only well-defined if there is one plant in the room, and a system that always acts on its first guess will act on the wrong guess often enough to matter. Ren et al. frame this formally: an LLM-based planner should recognise when it does not have enough information and ask for help rather than proceeding, and they give a taxonomy of the kinds of ambiguity (attribute, numeric, spatial) a household or office command tends to carry [2]. Second, language models hallucinate. They produce fluent, confident output that does not correspond to anything real, and Liang et al. show this gets worse, not better, as a task is broken into more steps, which is a direct problem for a robot expected to carry out multi-step instructions rather than single commands [3].

Both of these are usually studied against cloud-hosted, large-parameter models running on hardware that also has more sensing than a low-cost mobile robot typically carries. The rover used in this project for demonstration is the, a Hiwonder TurboPi. It does not fit that picture of large memory and compute power: a Raspberry Pi 5, a mecanum chassis, a camera on a 2-DOF pan/tilt gimbal, and no wheel encoders, IMU, or LiDAR. There is no odometry and no heading, and the mecanum wheels slip enough that an open-loop “drive forward two metres” cannot be trusted. Nothing on the Pi itself can run a language or vision model at a usable speed, and a stable internet connection cannot be assumed for a robot meant to operate on ordinary household or office connectivity. The pipeline built has to hold up under both constraints at once: ambiguity and hallucination have to be managed using models small enough to run locally on commodity hardware and not the large cloud-hosted models most of the literature above assumed.

The same idea, that a language model can reason about likely failure, also applies once a plan is already executing, not only while it is being made. Sinha et al. and Elhafi et al. both use an LLM as a runtime monitor, watching a robot’s observations for signs that something has gone wrong and triggering a safety response, for a quadrotor and for autonomous-driving and manipulation policies respectively [4, 5]. This project reuses that same principle for a rover reading its camera feed mid-mission, to decide whether a plan needs revising rather than only checking it once before departure.

1.2 PROBLEM STATEMENT

It is not established whether a fully local language-and-vision pipeline, with no dependency on a cloud API and small enough to run on a single consumer-grade machine, can reliably turn an ambiguous spoken command into a plan that a low-cost, sensor-poor indoor rover can actually carry out, or how much command complexity such a pipeline can absorb before it fails.

The literature reviewed in Chapter 2 addresses parts of this without closing the gap. Ren et al. and Liang et al. both address ambiguity and hallucination in LLM-based planning, but with large, cloud-hosted models and, in Ren et al.’s case, largely in simulated tabletop manipulation rather than on a physically constrained mobile platform [2, 3]. Hori et al. target long-horizon plans specifically but do not measure how many chained instructions a small, locally hosted model can carry before its output becomes unreliable [1]. Sinha et al. and Elhafi et al. monitor for failure once a task is already underway, but neither closes the loop back into a voice-driven clarification-and-confirmation step that gates a physical robot before it moves at all [4, 5].

This project addresses that gap directly, through two research questions. The first is whether locally hosted models, running entirely on a laptop-class machine paired with a rover that has no odometry or IMU, can generate plans that are both correct enough given

a vague command and executable given the rover’s actual capabilities. The second is how many commands can be chained together before the pipeline starts hallucinating: at what point does compositional depth outrun what a small, local model can reliably plan for.

1.3 RESEARCH QUESTIONS

This project is assessed against two research questions, both concerned with what a fully local speech-and-vision pipeline can and cannot do on hardware with no odometry or heading.

- RQ1** Can locally hosted models generate plans that a rover can actually execute and follow, even when the spoken command that produced them is vague? Answering this requires the plan to be correct enough despite an under specified command, and executable given what the rover can physically do.
- RQ2** How many commands can be chained together before the pipeline starts to hallucinate? This asks how far compositional depth can be pushed before the model’s output stops corresponding to anything the room, the command, or the rover’s capabilities actually support.

1.4 OBJECTIVES

The main objective of this project is to build and evaluate a fully local pipeline that turns a spoken, potentially ambiguous command into a plan an indoor rover can carry out, using models small enough to run on a single consumer-grade machine rather than a cloud API. To answer RQ1, this requires a multi-turn clarification loop that resolves ambiguity through spoken dialogue before a plan is generated, a planner constrained to a capability vocabulary the rover can actually perform, and a voice-only confirmation step that gates execution. To answer RQ2, the pipeline must log enough about each run, the number of chained instructions, the number of clarification turns used, and whether the resulting plan referenced anything never seen or said, to measure how compositional depth relates to hallucination.

A secondary objective is to demonstrate plan executability. Comparing the local pipeline against equivalent cloud-hosted models is a further, secondary objective, included as a benchmark rather than as evidence for either research question.

1.5 SCOPE AND LIMITATIONS

This project covers the local speech-to-execution pipeline: transcription, ambiguity resolution through spoken dialogue, plan generation and verification, and a voice-gated

confirmation step. Navigation is treated as instrumentation rather than a research problem in its own right.

The room the rover operates in is scanned once beforehand by a walkthrough video, producing a digest the planner can ground its plans in. Extending that scan into a general-purpose mapping system is not attempted. Cloud-hosted models appear only as a benchmarking comparison and are not part of the system under test.

The main limitation follows from timing. The rover itself has no wheel encoders, IMU, or LiDAR, so open-loop movement commands cannot be relied on for precision, a hardware limitation this project works around rather than solves. Evaluation is limited to a small number of test rooms and command sets, so the results characterise the pipeline’s behaviour on the cases tested rather than guaranteeing performance on arbitrary rooms or commands.

1.6 REPORT OUTLINE

Chapter 2 reviews existing work on LLM-based task planning, ambiguity resolution, and hallucination in robotic systems, and identifies where this project’s local, resource-constrained setting differs from it. Chapter 3 describes the system design: the speech-to-plan pipeline and its clarification loop. Chapter 4 covers the rover platform and the interface between the planning software and the hardware it controls. Chapter 5 describes the test methodology, including the simulated executor used in place of physical hardware. Chapters 6 and 7 present and discuss the results for RQ1 and RQ2, and Chapter 8 concludes with a summary and recommendations for future work.

LITERATURE STUDY

2.1 LANGUAGE MODELS AS ROBOT TASK PLANNERS

Large language models carry a great deal of general knowledge about the world, but that knowledge is not grounded in what any particular robot can actually do. An LLM asked to “tidy the kitchen” has plenty to say about what tidying a kitchen involves, and none of it accounts for whether the robot in front of it has an arm, what that arm can reach, or whether “the sponge” refers to something the robot has ever seen. To Close the gap, connecting the model’s reasoning to a specific robot’s actual capabilities, is what the systems in this section have in common, even though they close it in different ways.

SayCan [6] grounds the model by constraining its output rather than trusting it outright. The LLM proposes a set of candidate next actions drawn from a fixed library of pretrained skills, but each candidate is re-scored by a separately learned affordance function. that estimates how likely that skill is to succeed given the robot’s current state. The action that is both something the LLM would suggest and something the robot can plausibly do is the one that gets executed. Grounding, in this framing, is a second model checking the first, not something the LLM is trusted to know on its own.

Code as Policies [7] takes a different route. Rather than picking from a fixed skill library, the LLM writes code, calling perception functions, control primitives, and other functions it has been shown examples of, composed however the instruction requires. This lets it handle instructions no one anticipated in advance and assign precise parameters (a velocity, a threshold) to descriptions that were only ever given in words, at the cost of needing a language model capable of writing correct, composable code rather than just picking from a short list.

Inner Monologue [8] addresses a limitation both of the above still have: a plan committed to once, before anything happens, has no way to adapt as the situation unfolds. It closes the loop by feeding the robot’s ongoing observations, success and failure signals, scene descriptions, even a human’s spoken interjection, back into the language model’s own

reasoning process, so it can revise what it does next in light of what actually happened rather than what it expected to happen.

None of these three were built for tasks longer than about ten actions. Hori et al. [1] target that gap directly, arguing that long-horizon tasks fail for a different reason than short ones: the command usually leaves out information the model would need to plan the whole thing correctly, and no amount of re-scoring or code generation fixes a plan built on a wrong assumption. Their method resolves this through what they call active modification, the model asks clarifying questions before committing to a plan, combined with passive modification, refining the plan afterwards against visual feedback. This is the most direct ancestor of the clarification loop this project builds: extending plan length by first closing the information gap in the command, rather than by making the underlying model larger or its skill library richer.

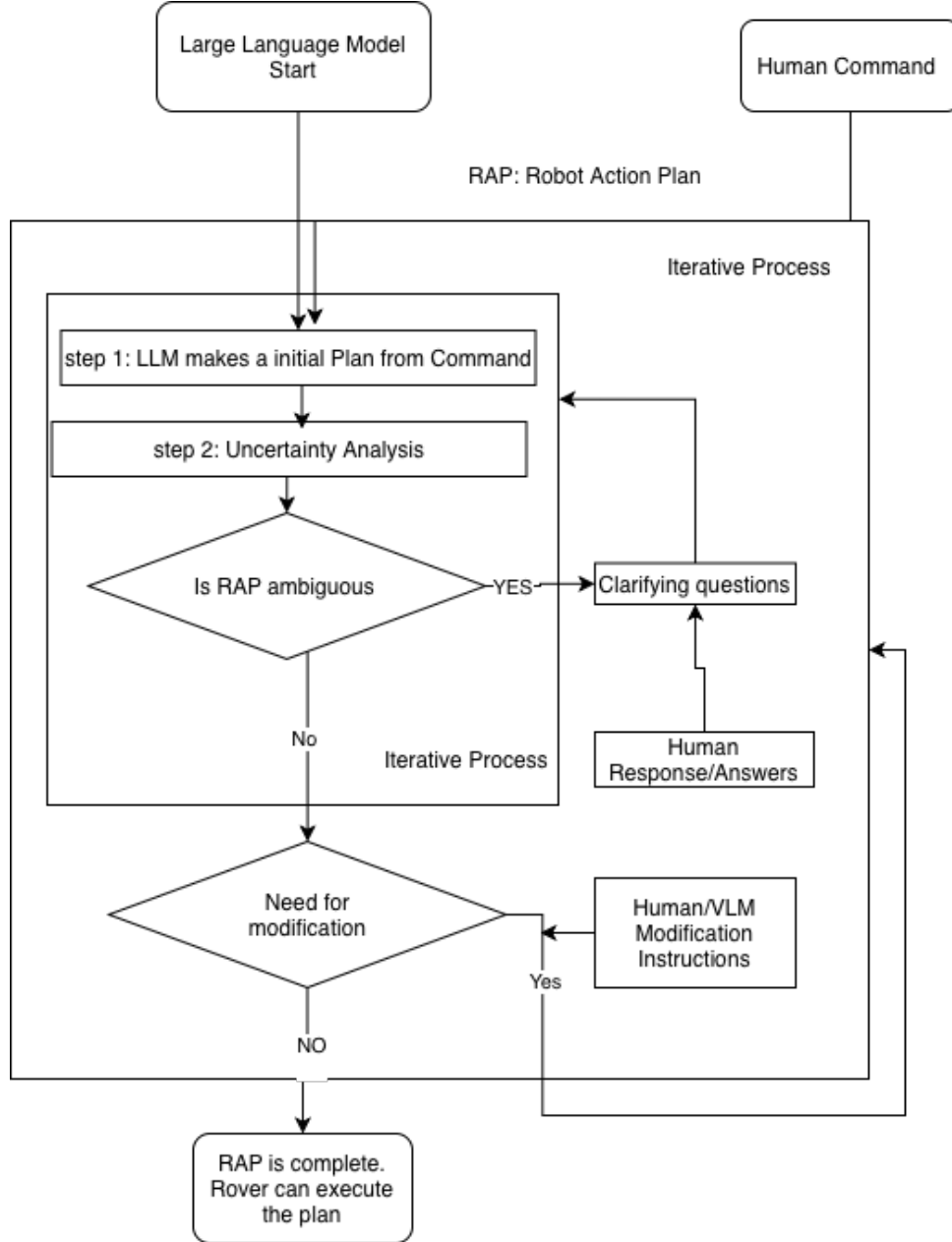


Figure 2.1: Process overview of RAP generation before rover departures

Last point Code as Policies’ approach assumes a language model capable of writing correct, composable code against a rich API, which is a reasonable assumption for a cloud-hosted model that has more computing power but not for the small, locally hosted planner this project depends on for RQ1.

2.2 GROUNDING PLANS IN A BOUNDED ACTION SET

SayCan and Code as Policies set the standard way to connect an LLM’s plan to what a robot can actually do. SayCan [6] pairs the LLM with a separately learned value function for each robot skill. The language model scores how relevant an action sounds for

the instruction. The value function scores whether the robot can do it right now. The two scores are combined, so the action picked is both sensible and physically possible. Figure 2.2 shows this with a concrete instruction, "how would you put an apple on the table?" The LLM alone ranks *find an apple*, *find a coke*, and *find a sponge* almost the same, since all three are plausible first steps for some task. The value function breaks that tie: *find an apple* and the other two score identically on the language side, but only the apple is actually visible in the robot's current scene, so the combined score, log-probability from the LLM multiplied against the affordance probability, pushes it to the top of the list. Nothing about the instruction told the model an apple was present. The value function is what supplied that.

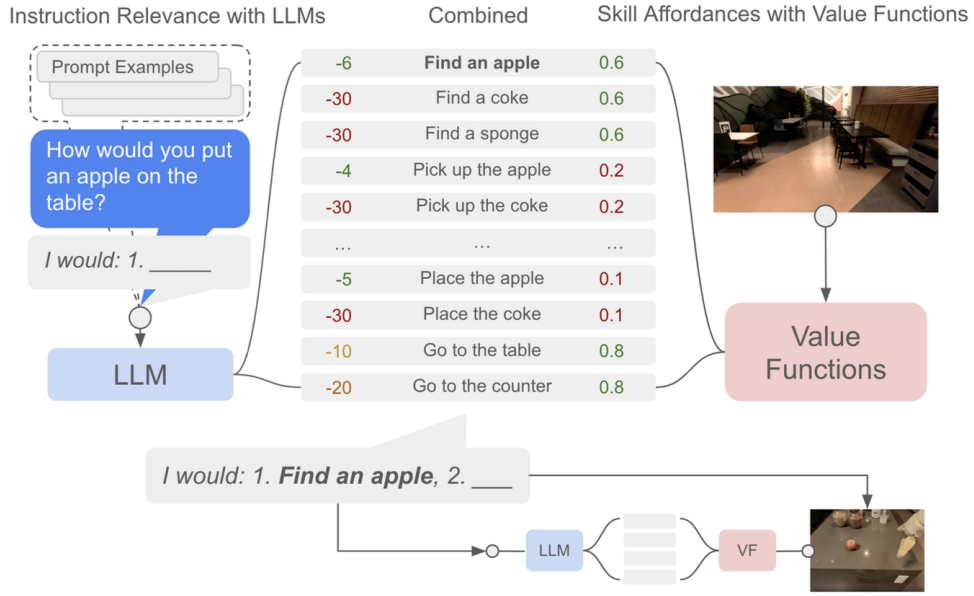


Figure 2.2: SayCan combining language-model relevance scores with value-function affordance scores to select the next skill, reproduced from Ahn et al. [6].

Ahn et al. formalise this selection rule as

$$\pi = \arg \max_{\pi \in \Pi} p(c_{\pi} \mid s, \ell_{\pi}) p(\ell_{\pi} \mid i) \quad (2.1)$$

where: Π is the set of available skill

ℓ_{π} is skill π 's textual label,

i is the instruction,

s is the current state,

c_{π} is the event that π completes successfully.

$p(\ell_{\pi} \mid i)$ is task-grounding, the language model's estimate of how well a skill matches the instruction, and $p(c_{\pi} \mid s, \ell_{\pi})$ is world-grounding, the value function's estimate of whether that skill would succeed from the current state.

These are exactly the two columns in Figure 2.2, the LLM’s log-probabilities are $\log p(\ell_\pi \mid i)$, and the 0.6/0.2/0.1/0.8 values are $p(c_\pi \mid s, \ell_\pi)$.

Code as Policies [7] solves the same problem a different way: the LLM writes Python that calls a fixed set of perception and control functions directly. If the code does not compile against that API, the step cannot run.

Both approaches give the planner a fixed set of actions it cannot go outside of. This project’s capability vocabulary in `planner.py` plays the same role: a step whose target or action falls outside that vocabulary is rejected rather than passed through to the rover.

Neither approach checks what happens after a step runs. Inner Monologue [8] adds that check on top of a SayCan-style setup: after each action, the robot gets an observation back, a success or failure signal, a scene description, or something the human said, and the next step is planned using that observation instead of the original plan alone. Plan a step, act, observe, replan. This project’s mid-mission execution follows the same loop: each frame sent over `/ws/execution` is what the plan gets checked against after each step, not only before the mission starts, allowing the plan to respond to drift the rover has actually accumulated rather than committing to a path decided before departure.

2.3 CLARIFICATION AND AMBIGUITY IN HUMAN-ROBOT INTERACTION

A spoken instruction only resolves to a single rover action if the rover’s observations rule out every other reading of it. "Bring the bowl" is unambiguous with one bowl on the counter and ambiguous with two. Hori et al. [1] describe two ways to correct a task plan once this kind of ambiguity shows up: passive modification, where a person reviews the plan and points out what is wrong, and active modification, where the planner judges its own uncertainty and asks a question before acting.

Passive modification is accurate, since a human checks every step, but it is repetitive. The plan has to be re-issued and re-checked each time something is off. Active modification moves that judgment onto the model, so the human is only asked to step in when the model is actually unsure.

Ren et al. [2] give active modification a statistical guarantee, in a method called KnowNo, and Hori et al. cite it directly as a working example of the approach. KnowNo does not let the planner commit to its single highest-scoring next step. The planner’s raw confidence scores for each candidate step are passed through a conformal prediction layer, which converts them into a calibrated set of steps that are all plausible enough to keep. A singleton set means the rover proceeds on its own. A set with more than one step means the instruction was ambiguous given what the rover can observe, and the rover asks a clarifying question instead of guessing. Figure 2.3 shows the example used in the paper.

It is told to place the bowl in the microwave with a metal and a plastic bowl both on the counter, the two matching steps score close enough that neither clears the calibrated threshold on its own, so the system asks which one, plastic or metal, instead of picking one.

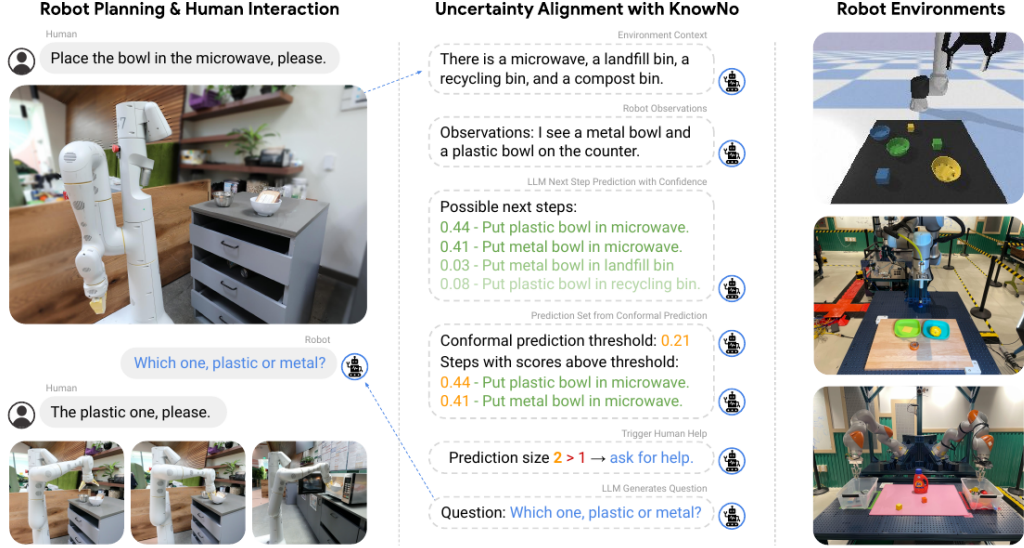


Figure 2.3: The KnowNo pipeline: an LLM planner proposes next steps with confidence scores, conformal prediction narrows these to a calibrated prediction set, and the rover asks for help whenever that set contains more than one option. Reproduced from Ren et al. [2].

The calibration works as follows. Given a held-out calibration set of N examples, each with a true label y_i and a model score $\hat{f}(\tilde{x}_i)_{y_i}$ for that label, a nonconformity score is computed for every example:

$$\kappa_i = 1 - \hat{f}(\tilde{x}_i)_{y_i}. \quad (2.2)$$

The calibrated threshold $\hat{q}$ is then set to the

$$\frac{\lceil (N+1)(1-\epsilon) \rceil}{N} \text{ empirical quantile of } \kappa_1, \dots, \kappa_N, \quad (2.3)$$

where ϵ is a user-chosen error tolerance. At test time, the prediction set kept for a new instruction is every label whose score clears this threshold:

$$C(\tilde{x}_{\text{test}}) = \{y \in Y \mid$$

$$f_{(\text{test})_y} \geq 1 - \hat{q}\} \quad (2.4)$$

Conformal prediction guarantees that this set contains the correct label with the user-specified probability,

$$P(y_{\text{test}} \in C(\tilde{x}_{\text{test}})) \geq 1 - \epsilon. \quad (2.5)$$

This guarantee is what makes the threshold usable in practice: instead of picking a confidence cut-off by hand and hoping it behaves sensibly, the rover’s decision to act alone comes with a stated bound on how often that decision is correct.

2.4 LOCALLY HOSTED AND QUANTISED MODELS

Sections 2.1 to 2.3 assume the rover can call out to a large, cloud-hosted LLM. A rover cannot always guarantee that access: network connectivity is not reliable in every test environment, and the round-trip latency to a cloud endpoint works against the response time a voice-commanded system needs to feel usable. Running the planner locally, on hardware the rover carries itself, removes both problems, but restricts the model to whatever fits in on-board memory.

Quantisation is the standard technique for making a model fit that constraint: model weights, and in some cases activations, are stored using fewer bits than the 16 or 32 bits used at training time, at some cost to numerical precision. Liu et al. [9] define uniform quantisation of a tensor $\mathbf{X}$ to b bits as

$$\hat{\mathbf{X}} = \mathcal{Q}(\mathbf{X}; b) = s \cdot \Pi_{\Omega(b)}\left(\frac{\mathbf{X}}{s}\right), \quad s = \frac{\max(\mathbf{X}) - \min(\mathbf{X})}{2^b - 1}, \quad (2.6)$$

where

* $\Omega(b) = \{0, 1, \dots, 2^b - 1\}$ is the set of representable integer levels

* $\Pi_{\Omega(b)}(\cdot)$ rounds to the nearest one.

This is applied at different bit-widths for weights and activations, written for example as W8A8 (8-bit weights, 8-bit activations) or W4A16 (4-bit weights, 16-bit activations); A lower bit-width gives a smaller model and a coarser step size s , and therefore more rounding error per value.

?] benchmark quantised LLMs directly on edge hardware and report that quantisation lowers energy use and inference latency substantially, at a small cost to output accuracy on straightforward tasks. Liu et al. [9] show that this cost is not constant across task types. Accuracy degrades faster as bit-width drops on tasks that require chaining several reasoning steps together than on single-step tasks. The planning step described in Section 2.1 is exactly that kind of multi-step reasoning task, so the bit-width chosen for a locally hosted planner in this project is not only an efficiency setting. It is a direct trade-off against how reliably the planner reasons through a multi-step voice command.

2.5 COMPOSITIONAL DEPTH AND HALLUCINATION

The methods discussed in Sections 2.1 to 2.3 treat each planning step largely on its own. As a task plan grows longer and more compositional, two further problems appear that a single-step clarification mechanism does not cover: the LLM’s reasoning can degrade over a long chain of steps, producing a confidently stated plan that is simply wrong, and the rover can encounter a scene it was never prepared for, one that no single faulty component causes but that the plan as a whole fails to account for.

Liang et al. [3] address the first problem by extending the conformal prediction procedure from Section 2.3 with a retrieval step. Rather than scoring each candidate plan from the LLM’s raw confidence alone, their method, introspective planning, first builds a knowledge base of worked examples. For a given task and observation, an LLM is prompted to explain in writing why certain candidate actions are valid and others are not. At deployment, the system retrieves the most similar past example by embedding similarity and includes it in the prompt before the LLM scores its candidates, as shown in Figure 2.4. Grounding each new decision in a similar, explained precedent sharpens the confidence scores that conformal prediction operates on, which produces the same statistically guaranteed prediction set from Equations 2.3–2.5 while asking the user fewer unnecessary questions.

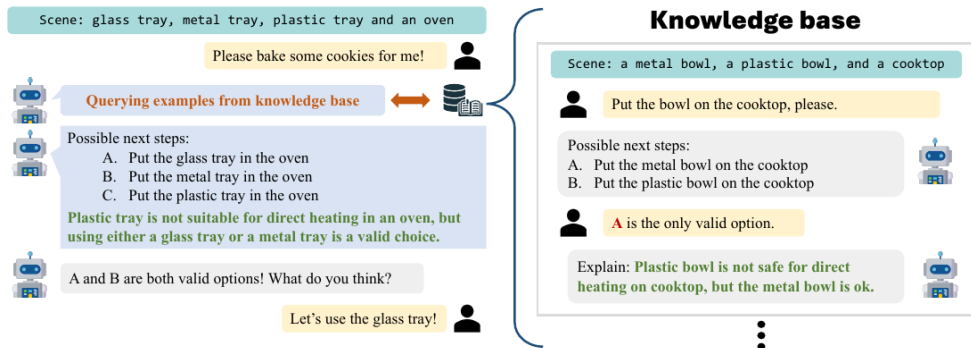


Figure 2.4: The introspective planning pipeline: a knowledge base of explained past examples is built offline, and at deployment the most similar example is retrieved to inform the LLM’s confidence for each candidate step. Reproduced from Liang et al. [3].

The second problem, an unanticipated scene rather than an unreliable plan, is addressed by Sinha et al. [4] and Elhafsi et al. [5] from the direction of runtime monitoring rather than planning. Sinha et al. propose a two-stage monitor: a fast binary classifier screens every observation in an LLM embedding space, and only when it flags a possible anomaly does a slower, more capable LLM reason explicitly about what has gone wrong and which fallback plan to switch to. Splitting the monitor this way keeps the fast path cheap enough to run continuously, while reserving the expensive reasoning step for the moments it is

actually needed, which matters directly for a rover with the limited on-board compute discussed in Section 2.4. Elhafsi et al. apply the same idea of an LLM as a monitor to what they term semantic anomalies. A system-level failures, like an autonomous vehicle braking for a stop sign printed on a billboard. That does not come from any single broken component, but from a gap in the system’s contextual reasoning.

Together, these three methods extend the single-step clarification of Section 2.3 along a different axis: introspective planning improves the confidence estimate feeding a single decision, while AESOP and semantic anomaly detection catch failures a confidence score never gets the chance to be wrong about, because the plan itself is being carried out against a scene the planner misjudged.

2.6 SPEECH RECOGNITION FOR SOUTH AFRICAN ENGLISH AND AFRIKAANS

Every method discussed so far assumes the instruction d_i handed to the LLM planner is already correct text. For a voice-commanded rover, that text comes from a speech-to-text stage first, and an error there becomes an error in the planner’s input before any clarification mechanism gets a chance to act on it. South African English and Afrikaans are both under-represented in the training data of general-purpose speech recognisers, which makes this an upstream risk specific to a South African test environment.

Kamper et al. [10] quantify how much accent variation costs a algorithmn used for recognising speech, trained without accounting for accent variation. Working with 31.78 hours of speech across five South African English accents (Afrikaans English, Black South African English, Cape Flats English, White South African English, and Indian South African English), they compare three acoustic modelling strategies: a separate model per accent, a single model pooling all accents together, and a multi-accent model that shares acoustic structure across accents where it helps while keeping accent-specific distinctions where it does not. The multi-accent model reached 82.78% word accuracy, against 81.53% for accent-specific models and 81.52% for a single pooled model, a difference the authors report as statistically significant. Black South African English stood out as the one accent that still benefited from being modelled separately rather than pooled, which the authors attribute to it being acoustically the most distinct of the five.

Jacobs et al. [11] work at the opposite end of the resource spectrum: fine-tuning Whisper on Afrikaans and isiXhosa oral narratives spoken by four- and five-year-old children, using only five minutes of transcribed in-domain child speech. They find that adding in-domain adult speech, especially combined with voice conversion to make it sound more child-like, gives the largest accuracy gain of any strategy tried, and that parameter-efficient fine-tuning helps Afrikaans but not isiXhosa, since isiXhosa is more

severely under-represented in Whisper’s original training data.

Together, these results point at the same conclusion from different directions. Local accents and languages measurably degrade off-the-shelf speech recognition, and the fix in both cases is not more general-purpose training data but a small, targeted amount of in-domain or in-accent data. For a rover meant to take spoken commands from South African English or Afrikaans speakers, this is a design constraint on the speech-to-text stage that sits upstream of every planning method discussed in Sections 2.1 to 2.5, not something those methods can correct for after the fact.

2.7 RESEARCH GAP

Sections 2.1 to 2.6 cover five separate strands of work: grounding an LLM’s plan in what a rover can actually do (Sections 2.1-2.2), asking for clarification when an instruction is genuinely ambiguous (Section 2.3), catching the failures that a confidence score alone will not (Section 2.5), running the planner on hardware small enough for a rover to carry (Section 2.4), and recognising speech accurately in a South African accent or language (Section 2.6). Each strand has been developed and evaluated largely on its own.

The methods in Sections 2.3 and 2.5 that provide statistical guarantees on plan correctness, KnowNo and introspective planning, are demonstrated with large, cloud-hosted LLMs. Section 2.4 shows that quantisation, needed to run a planner locally, costs accuracy disproportionately on exactly the kind of multi-step reasoning these planners perform, but neither Ren et al. [2] nor Liang et al. [3] evaluate their calibration procedure on a quantised, locally hosted model. Whether a conformal prediction threshold calibrated on a full-precision model still gives a valid guarantee once that model is quantised down to fit on-board memory is not addressed by either strand of work.

Similarly, none of the planning or clarification methods in Sections 2.1 to 2.5 are evaluated with speech input, let alone speech in a South African accent or in Afrikaans. Section 2.6 shows that accent and language mismatch measurably degrades transcription accuracy, which means an instruction d_i reaching the planner in this project’s setting carries a source of error the literature above does not model at all. An ambiguity introduced by a transcription error is not the same problem as an ambiguity in the underlying instruction, and it is unclear from the existing literature whether a clarification mechanism designed for the latter behaves sensibly when faced with the former.

This project sits at the intersection of these gaps: a voice-commanded indoor rover that must plan and clarify using a locally hosted, quantised LLM, taking speech input from South African English or Afrikaans speakers. No single work reviewed in this chapter evaluates this combination, and the remainder of this report investigates how the planning, clarification, and speech recognition components described above hold up once combined on hardware a rover runs on.

SYSTEM DESIGN

3.1 USER AND FUNCTIONAL REQUIREMENTS

3.2 SYSTEM ARCHITECTURE

3.2.1 OFF-BOARD INFERENCE

3.2.2 TWO-RATE CONTROL

3.3 THE CAPABILITY VOCABULARY

3.4 CLARIFICATION LOOP DESIGN

3.5 NAVIGATION AS INSTRUMENTATION

3.6 SUMMARY OF DESIGN DECISIONS

IMPLEMENTATION

- 4.1 SPEECH-TO-TEXT SUBSYSTEM
- 4.2 PLANNER
- 4.3 VISION-LANGUAGE VERIFICATION
- 4.4 CLARIFICATION MANAGER
- 4.5 ROVER CLIENT AND REACTIVE LAYER
- 4.6 COMMUNICATION PROTOCOL AND DASHBOARD
- 4.7 LOGGING AND MEASUREMENT INSTRUMENTATION

EXPERIMENTAL EVALUATION

5.1 EXPERIMENTAL METHODOLOGY

5.1.1 COMMAND CORPUS DESIGN

5.1.2 EVALUATION METRICS

5.2 PLAN CORRECTNESS UNDER VAGUE COMMANDS

5.3 EXECUTABILITY ON THE PHYSICAL CHASSIS

5.4 COMPOSITIONAL DEPTH AND HALLUCINATION ONSET

5.5 REFERENCE COMPARISON AGAINST CLOUD MODELS

5.6 DISCUSSION

CONCLUSION

6.1 ANSWERS TO THE RESEARCH QUESTIONS

6.2 CONTRIBUTIONS

6.3 LIMITATIONS

6.4 RECOMMENDATIONS FOR FUTURE WORK

REFERENCES

- [1] K. Hori, K. Suzuki, and T. Ogata, “Enhancement of long-horizon task planning via active and passive modification in large language models,” *Scientific Reports*, vol. 15, p. 7113, 2025.
- [2] A. Z. Ren, A. Dixit, A. Bodrova, S. Singh, S. Tu, N. Brown, P. Xu, L. Takayama, F. Xia, J. Varley, Z. Xu, D. Sadigh, A. Zeng, and A. Majumdar, “Robots that ask for help: Uncertainty alignment for large language model planners,” in *Proceedings of the 7th Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 229, 2023, arXiv:2307.01928.
- [3] K. Liang, Z. Zhang, and J. F. Fisac, “Introspective planning: Aligning robots’ uncertainty with inherent task ambiguity,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2402.06529.
- [4] R. Sinha, A. Elhafsi, C. Agia, M. Foutter, E. Schmerling, and M. Pavone, “Real-time anomaly detection and reactive planning with large language models,” in *Robotics: Science and Systems (RSS)*, 2024, arXiv:2407.08735.
- [5] A. Elhafsi, R. Sinha, C. Agia, E. Schmerling, I. A. D. Nesnas, and M. Pavone, “Semantic anomaly detection with large language models,” *Autonomous Robots*, vol. 47, no. 8, pp. 1035–1055, 2023.
- [6] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. Jauregui Ruano, K. Jeffrey, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng, “Do as I can, not as I say: Grounding language in robotic affordances,” in *Proceedings of the 6th Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 205, 2023, pp. 287–318, arXiv:2204.01691.
- [7] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as policies: Language model programs for embodied control,” in *Proceedings of*

- the IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 9493–9500.
- [8] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, N. Brown, T. Jackson, L. Luu, S. Levine, K. Hausman, and B. Ichter, “Inner monologue: Embodied reasoning through planning with language models,” in *Proceedings of the 6th Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 205, 2023, arXiv:2207.05608.
- [9] R. Liu, Y. Sun, M. Zhang, H. Bai, X. Yu, T. Yu, C. Yuan, and L. Hou, “Quantization hurts reasoning? an empirical study on quantized reasoning models,” 2025.
- [] E. J. Husom, A. Goknil, M. Astekin, L. K. Shar, A. Kåsen, S. Sen, B. A. Mithassel, and A. Soylu, “Sustainable LLM inference for edge AI: Evaluating quantized LLMs for energy efficiency, output accuracy, and inference latency,” *ACM Transactions on Internet of Things*, 2025.
- [10] H. Kamper, F. J. Muamba Mukanya, and T. R. Niesler, “Multi-accent acoustic modelling of south african english,” *Speech Communication*, vol. 54, no. 6, pp. 801–813, 2012.
- [11] C. Jacobs, A. Smith, D. Klop, O. Klejch, F. de Wet, and H. Kamper, “Speech recognition for automatically assessing Afrikaans and isiXhosa preschool oral narratives,” in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025.

APPENDIX A

APPENDIX TITLE GOES HERE
